



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра информационных технологий в атомной энергетике

ОТЧЕТ ПО ПРАКТИКЕ №6

«Автоматизированная система управления для обеспечения инвентарем для лыжного спорта»

по дисциплине «Автоматизированные системы управления элементами
промышленных и административных объектов»

Студент группы ИКБО-50-23

Павлов Н.С.

(подпись студента)

Принял преподаватель

Щербаков К. В.

(подпись руководителя)

Работа представлена

« ____ » _____ 2026 г.

Допущен к работе

« ____ » _____ 2026 г.

Москва 2026

ОГЛАВЛЕНИЕ

1. Анализ функций АСУ и определение функции для доработки	3
2. Анализ существующих языков программирования и IDE	5
3. Обоснованный выбор языка, IDE и библиотек	6
4. Генерация программного кода.....	8

1. Анализ функций АСУ и определение функции для доработки

Выбранная функция для разработки: Функция 1.7 — «Формирование плана закупок на период» (из Модуля 1: Управление закупками, планирование и бюджетный контроль).

Внутрисистемные функции, реализуемые модулем:

- 1.IS19 — Запуск алгоритма расчета потребности (анализ утвержденных заявок)
- 1.IS20 — Анализ текущих остатков на складе по каждой позиции
- 1.IS21 — Учет минимальных страховых запасов при расчете потребности
- 1.IS22 — Генерация проекта плана закупок с группировкой по номенклатуре

Обоснование выбора:

1. Высокая алгоритмическая сложность — прогнозирование требует применения методов машинного обучения (временные ряды, сезонность).

2. Специфичность предметной области — лыжный спорт имеет выраженную сезонность:

- Подготовка к зимнему сезону (август-октябрь)
- Пик использования (ноябрь-март)
- Межсезонное обслуживание (апрель-июль)

3. Учет множества факторов:

- Исторические данные по расходу инвентаря
- Сезонные коэффициенты спроса
- Нормативный износ лыж (зависит от интенсивности использования)
- Планируемые соревнования и сборы
- Рост/смена состава спортсменов (изменение размеров)

4. Экономическая значимость — ошибки прогнозирования приводят к:

- Дефициту (срыв тренировочного процесса)
- Избытку (заморозка бюджета, устаревание моделей)

5. Отсутствие готовых решений — типовые ERP-системы не учитывают специфику износа лыжного инвентаря.

2. Анализ существующих языков программирования и IDE

Таблица 2.1 – Рассмотренные языки программирования для задачи прогнозирования

Язык	Преимущества	Недостатки	Применимость к задаче
Python	Богатейшая экосистема для ML (pandas, scikit-learn, Prophet, statsmodels); быстрая разработка; интерактивный анализ в Jupyter	Динамическая типизация; ниже производительность на больших данных; GIL ограничивает многопоточность	Высокая — идеален для прототипирования и реализации ML-моделей прогнозирования
R	Специализирован для статистического анализа; отличная визуализация (ggplot2); множество пакетов для временных рядов	Сложный синтаксис; меньшее сообщество в РФ; сложнее интеграция с Java-бэкендом	Средняя — хорош для исследовательского анализа, но сложен в production
Java	Высокая производительность; строгая типизация; отличная интеграция с существующей архитектурой (Spring Boot)	Ограниченная поддержка ML-библиотек (Weka, Smile уступают Python); громоздкий код	Средняя — хорош для production-контура, но не для разработки моделей
Julia	Высокая производительность; синтаксис, близкий к математическому	Молодая экосистема; мало библиотек; сложность развертывания	Низкая — неоправданные риски для production

Таблица 2.2 – Рассмотренные IDE для разработки ML-модуля

IDE	Поддерживаемые языки	Преимущества	Недостатки
PyCharm Professional	Python, SQL, JavaScript	Лучшая поддержка научных библиотек; встроенный Jupyter Notebook; отладка DataFrame; интеграция с БД	Платная (бесплатна для студентов)
JupyterLab	Python, R, Julia	Интерактивная разработка; визуализация «на лету»; идеален для экспериментов	Не подходит для полноценного production-кода; нет рефакторинга
VS Code	Python, R, SQL	Бесплатный; отличные расширения для Python; поддержка Jupyter внутри редактора	Требует настройки окружения
DataSpell	Python, R, SQL	Специализированная IDE для Data Science от JetBrains; встроенный Jupyter	Платная; избыточна для небольшого модуля

3. Обоснованный выбор языка, IDE и библиотек

Выбранный язык программирования: Python 3.12

Обоснование:

- Библиотеки для временных рядов — Prophet (Facebook/Meta), statsmodels, scikit-learn — готовые реализации алгоритмов прогнозирования.
- Быстрое прототипирование — возможность проверить гипотезы в Jupyter Notebook перед интеграцией.
- Интеграция с Java-бэкендом — через REST API или экспорт модели в PMML/ONNX.
- Визуализация — matplotlib, seaborn, plotly для построения графиков прогнозов.
- Работа с данными — pandas для ETL-операций, загрузки из PostgreSQL.
- Соответствие архитектуре ПП5 — Python указан как язык аналитической подсистемы (позиция 2.4-2.5).

Выбранная IDE: PyCharm Professional 2026.1

Обоснование:

- Бесплатная лицензия для студентов — доступ к Professional версии.
- Встроенная поддержка Jupyter Notebook — можно вести разработку модели в .ipynb и затем экспортировать в .py.
- Интеграция с БД — просмотр таблиц PostgreSQL, выполнение SQL-запросов без переключения.
- Отладка DataFrame — просмотр содержимого pandas DataFrame в удобном табличном виде.
- Поддержка научных библиотек — автодополнение для pandas, numpy, sklearn.

Таблица 3.1 – Выбранные библиотеки Python

Библиотека	Версия	Назначение	Обоснование
pandas	2.2.0	Обработка и анализ данных	Стандарт для работы с табличными данными, загрузка из SQL
numpy	1.26.0	Численные вычисления	Базовые математические операции, работа с массивами
prophet	1.1.5	Прогнозирование временных рядов	Автоматический учет сезонности, праздников, трендов. Разработан Facebook специально для бизнес-прогнозирования
scikit-learn	1.4.0	Машинное обучение	Метрики качества прогноза (MAE, MAPE, RMSE), препроцессинг данных
statsmodels	0.14.1	Статистический анализ	SARIMA, экспоненциальное сглаживание, тесты на стационарность
psycopg2	2.9.9	Подключение к PostgreSQL	Загрузка исторических данных из БД АСУ
sqlalchemy	2.0.25	ORM для Python	Удобная работа с БД через pandas
matplotlib	3.8.2	Визуализация	Графики прогнозов, сезонных компонент
seaborn	0.13.2	Статистическая визуализация	Тепловые карты корреляций, улучшенные графики
plotly	5.18.0	Интерактивные графики	Дашборды для руководителя
holidays	0.40	Учет праздничных дней	Российские праздники влияют на работу склада
joblib	1.3.2	Сериализация моделей	Сохранение обученной модели для production
pydantic	2.5.0	Валидация данных	Проверка входных параметров прогноза
loguru	0.7.2	Логирование	Удобное логирование процесса прогнозирования
fastapi	0.108.0	REST API	Обертка для вызова прогноза из Java-бэкенда
uvicorn	0.25.0	ASGI-сервер	Запуск FastAPI приложения

4. Генерация программного кода

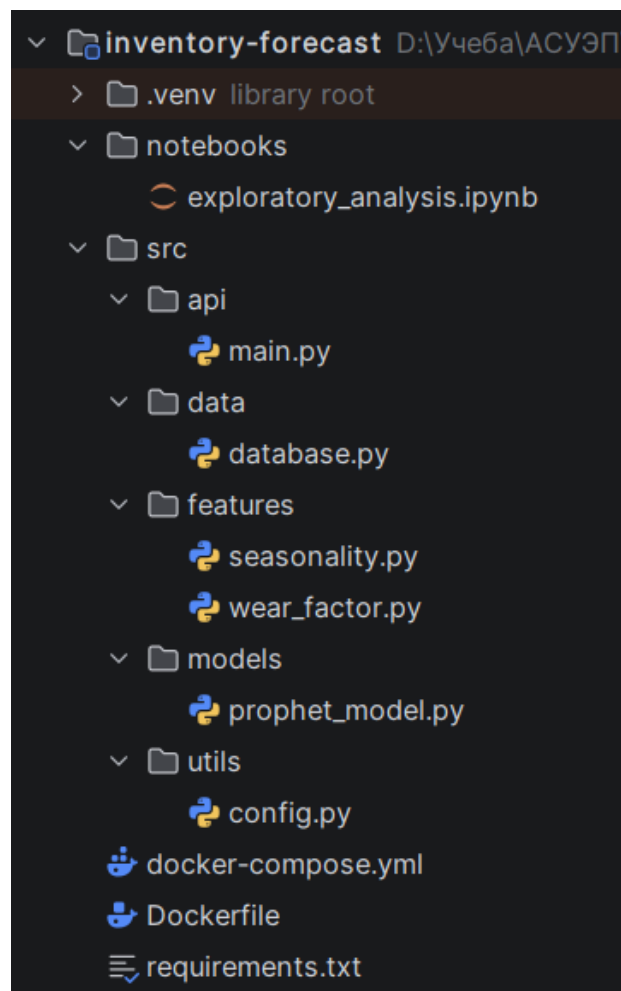


Рисунок 4.1 – Структура разрабатываемого модуля

Листинг 1 - Файл *requirements.txt*

```
# Core dependencies
pandas==2.2.0
numpy==1.26.0

# Forecasting libraries
prophet==1.1.5
scikit-learn==1.4.0
statsmodels==0.14.1

# Database
psycopg2-binary==2.9.9
sqlalchemy==2.0.25

# Visualization
matplotlib==3.8.2
seaborn==0.13.2
plotly==5.18.0

# Utilities
holidays==0.40
joblib==1.3.2
pydantic==2.5.0
```



```

loguru==0.7.2

# API
fastapi==0.108.0
uvicorn[standard]==0.25.0

# Development
jupyterlab==4.0.11
ipykernel==6.28.0
black==23.12.1
pytest==7.4.4

```

Листинг 2 – Файл `src/utils/config.py`

```

"""Конфигурация модуля прогнозирования."""
import os
from dataclasses import dataclass
from typing import Optional

@dataclass
class DatabaseConfig:
    """Настройки подключения к PostgreSQL."""
    host: str = os.getenv("DB_HOST", "localhost")
    port: int = int(os.getenv("DB_PORT", "5432"))
    database: str = os.getenv("DB_NAME", "asu_inventory")
    username: str = os.getenv("DB_USERNAME", "asu_user")
    password: str = os.getenv("DB_PASSWORD", "secure_password")

    @property
    def connection_string(self) -> str:
        return f"postgresql://{self.username}:{self.password}@{self.host}:{self.port}/{self.database}"

@dataclass
class ForecastConfig:
    """Настройки модели прогнозирования."""
    forecast_horizon_days: int = 90 # Прогноз на 3 месяца
    seasonality_mode: str = "multiplicative" # Мультипликативная сезонность
    для спорта
    yearly_seasonality: bool = True
    weekly_seasonality: bool = True
    daily_seasonality: bool = False
    changepoint_prior_scale: float = 0.05 # Гибкость тренда
    seasonality_prior_scale: float = 10.0 # Сила сезонности

    # Специфичные для лыжного спорта
    ski_wear_factor_enabled: bool = True # Учет износа лыж
    min_stock_days: int = 14 # Минимальный страховой запас в днях

@dataclass
class AppConfig:
    """Общая конфигурация приложения."""
    db: DatabaseConfig = DatabaseConfig()
    forecast: ForecastConfig = ForecastConfig()
    models_dir: str = os.getenv("MODELS_DIR", "./models")
    log_level: str = os.getenv("LOG_LEVEL", "INFO")

config = AppConfig()

```

Листинг 3 – Файл src/data/database.py

```
"""Модуль для работы с базой данных PostgreSQL."""
import pandas as pd
from sqlalchemy import create_engine, text
from loguru import logger
from typing import Optional, List, Dict, Any

from src.utils.config import config

class DatabaseManager:
    """Менеджер для работы с БД АСУ."""

    def __init__(self):
        self.engine = create_engine(
            config.db.connection_string,
            pool_size=5,
            max_overflow=10,
            echo=False
        )
        logger.info(f"Подключение к БД: {config.db.host}:{config.db.port}/{config.db.database}")

    def get_historical_consumption(
        self,
        nomenclature_id: int,
        start_date: str,
        end_date: Optional[str] = None
    ) -> pd.DataFrame:
        """
        Загрузка исторических данных по расходу инвентаря.

        Args:
            nomenclature_id: ID номенклатуры (тип инвентаря)
            start_date: Начальная дата в формате 'YYYY-MM-DD'
            end_date: Конечная дата (None = текущая дата)

        Returns:
            DataFrame с колонками: ds (дата), y (количество выданного инвентаря)
        """
        query = """
        SELECT
            DATE(d.issuance_date) as ds,
            COUNT(*) as y
        FROM issuance_documents d
        JOIN inventory_items i ON d.inventory_item_id = i.id
        WHERE i.nomenclature_id = :nomenclature_id
            AND d.issuance_date >= :start_date
        """
        params = {"nomenclature_id": nomenclature_id, "start_date": start_date}

        if end_date:
            query += " AND d.issuance_date <= :end_date"
            params["end_date"] = end_date

        query += " GROUP BY DATE(d.issuance_date) ORDER BY ds"

        logger.info(f"Загрузка исторических данных для номенклатуры {nomenclature_id}")

        with self.engine.connect() as conn:
            df = pd.read_sql(text(query), conn, params=params)
```

```

        # Заполняем пропущенные даты нулями
        if not df.empty:
            df['ds'] = pd.to_datetime(df['ds'])
            date_range = pd.date_range(start=df['ds'].min(),
end=df['ds'].max())
            df = df.set_index('ds').reindex(date_range, fill_value=0).re-
set_index()
            df.columns = ['ds', 'y']

        logger.info(f"Загружено {len(df)} записей")
        return df

    def get_current_stock(self, nomenclature_id: int) -> int:
        """Получение текущего остатка на складе."""
        query = """
        SELECT COUNT(*) as stock
        FROM inventory_items
        WHERE nomenclature_id = :nomenclature_id
          AND status = 'AVAILABLE'
        """
        with self.engine.connect() as conn:
            result = conn.execute(text(query), {"nomenclature_id": nomenclature_id})
            return result.scalar() or 0

    def get_nomenclature_info(self, nomenclature_id: int) -> Dict[str, Any]:
        """Получение информации о номенклатуре."""
        query = """
        SELECT
            id,
            name,
            category,
            min_stock_level,
            standard_service_life_months,
            manufacturer
        FROM nomenclature
        WHERE id = :nomenclature_id
        """
        with self.engine.connect() as conn:
            result = conn.execute(text(query), {"nomenclature_id": nomenclature_id})
            row = result.fetchone()
            if row:
                return dict(row._mapping)
            return {}

    def get_athletes_count_by_size(self, nomenclature_id: int) -> pd.DataFrame:
        """
        Получение распределения спортсменов по размерам.
        Для прогнозирования потребности в конкретных ростовках.
        """
        query = """
        SELECT
            i.size,
            COUNT(DISTINCT a.id) as athletes_count
        FROM athletes a
        CROSS JOIN nomenclature n
        LEFT JOIN inventory_items i ON i.nomenclature_id = n.id
          AND i.status = 'ISSUED'
          AND i.current_holder_id = a.id
        WHERE n.id = :nomenclature_id
        GROUP BY i.size

```

```

        """
        with self.engine.connect() as conn:
            return pd.read_sql(text(query), conn, params={"nomenclature_id":
nomenclature_id})

    def get_upcoming_events(self, days_ahead: int = 90) -> pd.DataFrame:
        """
        Получение списка предстоящих соревнований и сборов.
        Влияет на пиковый спрос.
        """
        query = """
        SELECT
            event_date,
            event_type,
            expected_participants,
            equipment_needed
        FROM events
        WHERE event_date BETWEEN CURRENT_DATE AND CURRENT_DATE + :days_ahead
        ORDER BY event_date
        """
        with self.engine.connect() as conn:
            return pd.read_sql(text(query), conn, params={"days_ahead":
days_ahead})

```

Листинг 4 – Файл src/features/seasonality.py

```

"""Расчет сезонных коэффициентов для лыжного спорта."""
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
from typing import Dict, List, Tuple
from loguru import logger

class SkiSeasonalityCalculator:
    """
    Калькулятор сезонности для лыжного спорта.

    Учитывает:
    - Подготовительный период (август-октябрь): закупка инвентаря
    - Соревновательный сезон (ноябрь-март): пиковое использование
    - Межсезонье (апрель-июль): обслуживание и ремонт
    """

    # Месячные коэффициенты спроса (на основе исторических данных)
    MONTHLY_FACTORS = {
        1: 1.2, # Январь - пик сезона
        2: 1.3, # Февраль - пик сезона
        3: 1.1, # Март - завершение сезона
        4: 0.4, # Апрель - межсезонье
        5: 0.2, # Май - межсезонье
        6: 0.1, # Июнь - минимум
        7: 0.1, # Июль - минимум
        8: 0.5, # Август - начало подготовки
        9: 0.8, # Сентябрь - активная подготовка
        10: 1.0, # Октябрь - предсезонные сборы
        11: 1.1, # Ноябрь - начало соревнований
        12: 1.2 # Декабрь - разгар сезона
    }

    @classmethod
    def get_monthly_factor(cls, month: int) -> float:
        """Получить коэффициент для месяца."""

```

```

        return cls.MONTHLY_FACTORS.get(month, 1.0)

    @classmethod
    def create_seasonality_features(cls, dates: pd.DatetimeIndex) -> pd.DataFrame:
        """
        Создание признаков сезонности для модели.

        Args:
            dates: Индекс с датами

        Returns:
            DataFrame с признаками: month, is_winter_season, is_preparation_period
        """
        df = pd.DataFrame(index=dates)
        df['month'] = dates.month
        df['month_factor'] = df['month'].map(cls.MONTHLY_FACTORS)
        df['is_winter_season'] = df['month'].isin([11, 12, 1, 2, 3]).astype(int)
        df['is_preparation_period'] = df['month'].isin([8, 9, 10]).astype(int)
        df['is_off_season'] = df['month'].isin([4, 5, 6, 7]).astype(int)

        # День недели (выдача чаще в начале недели)
        df['day_of_week'] = dates.dayofweek
        df['is_weekend'] = (df['day_of_week'] >= 5).astype(int)

        return df

    @classmethod
    def adjust_forecast_by_seasonality(
        cls,
        forecast: pd.DataFrame,
        seasonality_multiplier: float = 1.0
    ) -> pd.DataFrame:
        """
        Корректировка прогноза с учетом сезонности.

        Args:
            forecast: Прогноз с колонкой 'ds' (дата)
            seasonality_multiplier: Множитель сезонности (для усиления/ослабления)

        Returns:
            Скорректированный прогноз
        """
        forecast = forecast.copy()
        forecast['month'] = pd.to_datetime(forecast['ds']).dt.month
        forecast['seasonality_factor'] = forecast['month'].map(cls.MONTHLY_FACTORS)

        # Нормализация факторов
        mean_factor = np.mean(list(cls.MONTHLY_FACTORS.values()))
        forecast['seasonality_factor'] = forecast['seasonality_factor'] / mean_factor

        # Применение к прогнозу
        forecast['yhat'] = forecast['yhat'] * (1 + (forecast['seasonality_factor'] - 1) * seasonality_multiplier)
        forecast['yhat_lower'] = forecast['yhat_lower'] * (
            1 + (forecast['seasonality_factor'] - 1) * seasonality_multiplier)
        forecast['yhat_upper'] = forecast['yhat_upper'] * (

```

```

        1 + (forecast['seasonality_factor'] - 1) * seasonal-
ity_multiplier)

    return forecast

```

Листинг 5 – Файл `src/features/wear_factor.py`

```

"""Расчет коэффициента износа лыжного инвентаря."""
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
from typing import Dict, Optional
from loguru import logger
from sqlalchemy import text

class SkiWearCalculator:
    """
    Калькулятор износа лыжного инвентаря.

    Учитывает:
    - Нормативный срок службы (из карточки номенклатуры)
    - Интенсивность использования (количество выдач)
    - Сезонность использования
    - Плановую замену инвентаря
    """

    def __init__(self, db_manager):
        self.db_manager = db_manager

    def calculate_wear_rate(
        self,
        nomenclature_id: int,
        forecast_horizon_days: int
    ) -> float:
        """
        Расчет ожидаемого процента износа за период прогноза.

        Returns:
        Коэффициент износа (0-1) - доля инвентаря, требующая замены
        """
        info = self.db_manager.get_nomenclature_info(nomenclature_id)
        if not info:
            logger.warning(f"Номенклатура {nomenclature_id} не найдена")
            return 0.05 # Значение по умолчанию 5% в квартал

        standard_life_months = info.get('standard_service_life_months', 24)

        # Запрос среднего возраста инвентаря
        query = """
        SELECT
            AVG(EXTRACT(DAY FROM NOW() - commissioning_date)) / 30.0 as
avg_age_months,
            COUNT(*) as total_items,
            COUNT(CASE
                WHEN EXTRACT(MONTH FROM AGE(NOW(), commissioning_date)) >
:standard_life_months
            THEN 1
            END) as expired_items
        FROM inventory_items
        WHERE nomenclature_id = :nomenclature_id
        AND status NOT IN ('WRITTEN_OFF', 'DEFECTIVE')
        """

```

```

        with self.db_manager.engine.connect() as conn:
            result = conn.execute(
                text(query),
                {"nomenclature_id": nomenclature_id, "standard_life_months":
standard_life_months}
            )
            row = result.fetchone()

            if row and row.total_items > 0:
                avg_age_months = row.avg_age_months or 0
                expired_ratio = row.expired_items / row.total_items

                # Прогнозируемый износ за период
                forecast_months = forecast_horizon_days / 30.0
                forecast_wear = (avg_age_months + forecast_months) / stand-
ard_life_months
                forecast_wear = np.clip(forecast_wear, 0.05, 0.5) # Минимум
5%, максимум 50%

                logger.info(f"Средний возраст: {avg_age_months:.1f} мес, "
                            f"просрочено: {expired_ratio:.1%}, "
                            f"прогноз износа: {forecast_wear:.1%}")

                return max(forecast_wear, expired_ratio)

        return 0.05

def calculate_replacement_need(
    self,
    nomenclature_id: int,
    forecasted_demand: float,
    forecast_horizon_days: int
) -> Dict[str, float]:
    """
    Расчет полной потребности в закупке с учетом износа.

    Returns:
        Словарь с компонентами потребности:
        - base_demand: базовый спрос
        - wear_replacement: замена изношенного инвентаря
        - safety_stock: страховой запас
        - total_need: итоговая потребность
    """
    current_stock = self.db_manager.get_current_stock(nomenclature_id)
    wear_rate = self.calculate_wear_rate(nomenclature_id, forecast_hori-
zon_days)

    # Оценка общего количества инвентаря в обращении
    query = """
    SELECT COUNT(*) as total_active
    FROM inventory_items
    WHERE nomenclature_id = :nomenclature_id
    AND status IN ('AVAILABLE', 'ISSUED', 'RESERVED')
    """
    with self.db_manager.engine.connect() as conn:
        result = conn.execute(text(query), {"nomenclature_id": nomencla-
ture_id})
        total_active = result.scalar() or 0

    # Расчет компонентов
    base_demand = forecasted_demand
    wear_replacement = total_active * wear_rate

```

```

info = self.db_manager.get_nomenclature_info(nomenclature_id)
min_stock_level = info.get('min_stock_level', 5)
safety_stock = max(min_stock_level - current_stock, 0)

total_need = base_demand + wear_replacement + safety_stock
total_need = max(total_need, 0)

return {
    'base_demand': round(base_demand, 1),
    'wear_replacement': round(wear_replacement, 1),
    'safety_stock': round(safety_stock, 1),
    'current_stock': current_stock,
    'total_active': total_active,
    'wear_rate': round(wear_rate, 3),
    'total_need': round(total_need, 1)
}

```

Листинг 6 – Файл `src/models/prophet_model.py`

```

"""Модель прогнозирования на основе Facebook Prophet."""
import pandas as pd
import numpy as np
from prophet import Prophet
from loguru import logger
import joblib
from typing import Optional, Dict, Any, Tuple
from datetime import datetime, timedelta

from src.utils.config import config
from src.features.seasonality import SkiSeasonalityCalculator

class SkiInventoryForecastModel:
    """
    Модель прогнозирования потребности в лыжном инвентаре.

    Использует Prophet с дополнительными настройками для учета:
    - Сезонности лыжного спорта
    - Российских праздников
    - Предстоящих соревнований
    """

    def __init__(self, model_name: str = "ski_forecast"):
        self.model_name = model_name
        self.model: Optional[Prophet] = None
        self.seasonality_calculator = SkiSeasonalityCalculator()
        self.training_metrics: Dict[str, float] = {}

    def _create_model(self) -> Prophet:
        """Создание и настройка модели Prophet."""
        model = Prophet(
            growth='linear',
            seasonality_mode=config.forecast.seasonality_mode,
            yearly_seasonality=config.forecast.yearly_seasonality,
            weekly_seasonality=config.forecast.weekly_seasonality,
            daily_seasonality=config.forecast.daily_seasonality,
            changepoint_prior_scale=config.forecast.changepoint_prior_scale,
            seasonality_prior_scale=config.forecast.seasonality_prior_scale,
            interval_width=0.95 # 95% доверительный интервал
        )

        # Добавление российских праздников
        model.add_country_holidays(country_name='RU')

```



```

# Добавление специфичной сезонности для лыжного спорта
model.add_seasonality(
    name='winter_season',
    period=365.25,
    fourier_order=5,
    prior_scale=15.0,
    condition_name='is_winter_season'
)

model.add_seasonality(
    name='preparation_period',
    period=365.25,
    fourier_order=3,
    prior_scale=10.0,
    condition_name='is_preparation_period'
)

return model

def prepare_data(
    self,
    df: pd.DataFrame,
    include_seasonality_features: bool = True
) -> pd.DataFrame:
    """
    Подготовка данных для обучения.

    Args:
        df: DataFrame с колонками ds (дата) и y (значение)
        include_seasonality_features: Добавлять ли признаки сезонности

    Returns:
        Подготовленный DataFrame
    """
    df = df.copy()
    df['ds'] = pd.to_datetime(df['ds'])

    if include_seasonality_features:
        features = self.seasonality_calculator.create_seasonality_features(df['ds'])
        df = pd.concat([df, features], axis=1)

    # Удаление выбросов (метод IQR)
    Q1 = df['y'].quantile(0.25)
    Q3 = df['y'].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    df = df[(df['y'] >= lower_bound) & (df['y'] <= upper_bound)]

    logger.info(f"Данные подготовлены: {len(df)} записей")
    return df

def fit(self, df: pd.DataFrame, **kwargs) -> 'SkiInventoryForecastModel':
    """
    Обучение модели на исторических данных.

    Args:
        df: DataFrame с колонками ds и y
        **kwargs: Дополнительные параметры для Prophet.fit()

    Returns:

```

```

        self
        """
        logger.info(f"Начало обучения модели {self.model_name}")

        df = self.prepare_data(df)

        if len(df) < 30:
            logger.warning(f"Недостаточно данных для обучения: {len(df)} за-
писей")

            # Используем простую скользящую среднюю
            self._create_simple_model(df)
            return self

        self.model = self._create_model()
        self.model.fit(df, **kwargs)

        # Расчет метрик на обучающей выборке
        forecast = self.model.predict(df)
        self._calculate_metrics(df['y'].values, forecast['yhat'].val-
ues[:len(df)])

        logger.info(f"Модель обучена. MAE: {self.training_metrics.get('mae',
0):.2f}")
        return self

    def _create_simple_model(self, df: pd.DataFrame):
        """Создание простой модели при недостатке данных."""
        self.model = None
        self._simple_mean = df['y'].mean()
        self._simple_std = df['y'].std()
        logger.info(f"Создана простая модель: mean={self._simple_mean:.2f},
std={self._simple_std:.2f}")

    def predict(self, periods: int = 90) -> pd.DataFrame:
        """
        Прогнозирование на будущие периоды.

        Args:
            periods: Количество дней для прогноза

        Returns:
            DataFrame с прогнозом: ds, yhat, yhat_lower, yhat_upper
        """
        if self.model is None:
            # Используем простую модель
            future_dates = pd.date_range(
                start=datetime.now(),
                periods=periods,
                freq='D'
            )
            return pd.DataFrame({
                'ds': future_dates,
                'yhat': self._simple_mean,
                'yhat_lower': max(0, self._simple_mean - 2 * self._sim-
ple_std),
                'yhat_upper': self._simple_mean + 2 * self._simple_std
            })

        future = self.model.make_future_dataframe(periods=periods)

        # Добавление признаков сезонности
        features = self.seasonality_calculator.create_seasonality_fea-
tures(future['ds'])
        future = pd.concat([future, features], axis=1)

```

```

forecast = self.model.predict(future)

# Корректировка с учетом сезонности лыжного спорта
forecast = self.seasonality_calculator.adjust_forecast_by_seasonality(
    forecast, seasonality_multiplier=1.2
)

# Отрицательные значения заменяем на 0
forecast['yhat'] = forecast['yhat'].clip(lower=0)
forecast['yhat_lower'] = forecast['yhat_lower'].clip(lower=0)
forecast['yhat_upper'] = forecast['yhat_upper'].clip(lower=0)

return forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']]

def predict_demand(
    self,
    periods: int = 90
) -> Dict[str, Any]:
    """
    Прогнозирование суммарного спроса на период.

    Returns:
        Словарь с суммарным прогнозом и доверительными интервалами
    """
    forecast = self.predict(periods)

    total_demand = forecast['yhat'].sum()
    total_lower = forecast['yhat_lower'].sum()
    total_upper = forecast['yhat_upper'].sum()

    return {
        'forecast_period_days': periods,
        'total_demand': round(total_demand, 1),
        'confidence_interval': {
            'lower': round(total_lower, 1),
            'upper': round(total_upper, 1)
        },
        'daily_forecast': forecast.to_dict('records')
    }

def _calculate_metrics(self, y_true: np.ndarray, y_pred: np.ndarray):
    """Расчет метрик качества прогноза."""
    from sklearn.metrics import mean_absolute_error, mean_squared_error

    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))

    # MAPE (Mean Absolute Percentage Error)
    mape = np.mean(np.abs((y_true - y_pred) / np.where(y_true == 0, 1,
y_true))) * 100

    self.training_metrics = {
        'mae': mae,
        'rmse': rmse,
        'mape': mape
    }

def save(self, path: Optional[str] = None):
    """Сохранение модели."""
    import os
    if path is None:
        path = os.path.join(config.models_dir, f"{self.model_name}.pkl")

```

```

os.makedirs(os.path.dirname(path), exist_ok=True)

model_data = {
    'model': self.model,
    'model_name': self.model_name,
    'training_metrics': self.training_metrics,
    'config': config.forecast
}
joblib.dump(model_data, path)
logger.info(f"Модель сохранена: {path}")

@classmethod
def load(cls, path: str) -> 'SkiInventoryForecastModel':
    """Загрузка модели."""
    model_data = joblib.load(path)

    instance = cls(model_name=model_data['model_name'])
    instance.model = model_data['model']
    instance.training_metrics = model_data['training_metrics']

    logger.info(f"Модель загружена: {path}")
    return instance

```

Листинг 7 – Файл `src/api/main.py` (FastAPI приложение)

```

"""REST API для модуля прогнозирования."""
from fastapi import FastAPI, HTTPException, Query
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, Field
from typing import Optional, List, Dict, Any
from datetime import datetime, timedelta
from loguru import logger
import uvicorn

from src.data.database import DatabaseManager
from src.models.prophet_model import SkiInventoryForecastModel
from src.features.wear_factor import SkiWearCalculator
from src.utils.config import config

# Pydantic схемы
class ForecastRequest(BaseModel):
    """Запрос на прогнозирование."""
    nomenclature_id: int = Field(..., description="ID номенклатуры")
    forecast_days: int = Field(90, ge=7, le=365, description="Горизонт прогноза в днях")
    include_wear: bool = Field(True, description="Учитывать износ инвентаря")

    class Config:
        json_schema_extra = {
            "example": {
                "nomenclature_id": 123,
                "forecast_days": 90,
                "include_wear": True
            }
        }

class ForecastResponse(BaseModel):
    """Ответ с прогнозом."""
    nomenclature_id: int

```

```

nomenclature_name: str
forecast_period_days: int
forecast_start_date: str
forecast_end_date: str
base_demand: float
wear_replacement: float
safety_stock: float
current_stock: int
total_need: float
recommended_order: float
confidence_interval: Dict[str, float]
daily_forecast: List[Dict[str, Any]]
model_metrics: Dict[str, float]
created_at: str

class ModelInfoResponse(BaseModel):
    """Информация о модели."""
    model_name: str
    is_trained: bool
    training_metrics: Dict[str, float]
    last_trained: Optional[str]

class TrainRequest(BaseModel):
    """Запрос на обучение модели."""
    nomenclature_id: int = Field(..., description="ID номенклатуры")
    historical_days: int = Field(365, ge=90, le=1095, description="Дней истории для обучения")

# Создание приложения
app = FastAPI(
    title="ASU Ski Inventory Forecast API",
    description="API для прогнозирования потребности в лыжном инвентаре",
    version="1.0.0"
)

# CORS для доступа из веб-интерфейса
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Глобальные объекты
db_manager = DatabaseManager()
wear_calculator = SkiWearCalculator(db_manager)
models_cache: Dict[int, SkiInventoryForecastModel] = {}

def get_or_train_model(nomenclature_id: int, force_retrain: bool = False) -> SkiInventoryForecastModel:
    """Получение или обучение модели для номенклатуры."""
    import os
    model_path = os.path.join(config.models_dir, f"model_{nomenclature_id}.pkl")

    if not force_retrain and os.path.exists(model_path):
        return SkiInventoryForecastModel.load(model_path)

# Загрузка исторических данных

```

```

start_date = (datetime.now() - timedelta(days=365)).strftime('%Y-%m-%d')
df = db_manager.get_historical_consumption(nomenclature_id, start_date)

if df.empty:
    raise HTTPException(status_code=404, detail=f"Нет исторических дан-
ных для номенклатуры {nomenclature_id}")

# Обучение модели
model = SkiInventoryForecastModel(model_name=f"ski_forecast_{nomencla-
ture_id}")
model.fit(df)
model.save(model_path)

models_cache[nomenclature_id] = model
return model

@app.get("/health")
async def health_check():
    """Проверка работоспособности."""
    return {"status": "healthy", "timestamp": datetime.now().isoformat()}

@app.post("/forecast", response_model=ForecastResponse)
async def create_forecast(request: ForecastRequest):
    """
    Создание прогноза потребности в закупке.

    Возвращает детальный прогноз с разбивкой по компонентам:
    - Базовый спрос (на основе исторических данных)
    - Замена изношенного инвентаря
    - Страховой запас
    """
    logger.info(f"Запрос прогноза: nomenclature_id={request.nomencla-
ture_id}, days={request.forecast_days}")

    try:
        # Получение информации о номенклатуре
        info = db_manager.get_nomenclature_info(request.nomenclature_id)
        if not info:
            raise HTTPException(status_code=404, detail=f"Номенклатура {re-
quest.nomenclature_id} не найдена")

        # Получение или обучение модели
        model = get_or_train_model(request.nomenclature_id)

        # Прогноз спроса
        demand_forecast = model.predict_demand( periods=request.fore-
cast_days)
        base_demand = demand_forecast['total_demand']

        # Расчет износа и полной потребности
        if request.include_wear:
            replacement_need = wear_calculator.calculate_replacement_need(
                request.nomenclature_id,
                base_demand,
                request.forecast_days
            )
        else:
            current_stock = db_manager.get_current_stock(request.nomencla-
ture_id)
            replacement_need = {
                'base_demand': base_demand,
                'wear replacement': 0,
            }

```

```

        'safety_stock': max(0, info.get('min_stock_level', 5) - current_stock),
        'current_stock': current_stock,
        'total_active': 0,
        'wear_rate': 0,
        'total_need': base_demand + max(0, info.get('min_stock_level', 5) - current_stock)
    }

    # Рекомендуемый заказ (с учетом текущего остатка)
    recommended_order = max(0, replacement_need['total_need'] - replacement_need['current_stock'])

    start_date = datetime.now().date()
    end_date = start_date + timedelta(days=request.forecast_days)

    return ForecastResponse(
        nomenclature_id=request.nomenclature_id,
        nomenclature_name=info.get('name', 'Unknown'),
        forecast_period_days=request.forecast_days,
        forecast_start_date=start_date.isoformat(),
        forecast_end_date=end_date.isoformat(),
        base_demand=round(replacement_need['base_demand'], 1),
        wear_replacement=round(replacement_need['wear_replacement'], 1),
        safety_stock=round(replacement_need['safety_stock'], 1),
        current_stock=replacement_need['current_stock'],
        total_need=round(replacement_need['total_need'], 1),
        recommended_order=round(recommended_order, 1),
        confidence_interval=demand_forecast['confidence_interval'],
        daily_forecast=demand_forecast['daily_forecast'][-request.forecast_days:],
        model_metrics=model.training_metrics,
        created_at=datetime.now().isoformat()
    )

except HTTPException:
    raise
except Exception as e:
    logger.error(f"Ошибка при создании прогноза: {e}")
    raise HTTPException(status_code=500, detail=str(e))

@app.post("/train", response_model=ModelInfoResponse)
async def train_model(request: TrainRequest):
    """
    Обучение модели на исторических данных.

    Используется для первоначального обучения или переобучения
    при накоплении новых данных.
    """
    logger.info(f"Запрос обучения модели: nomenclature_id={request.nomenclature_id}")

    try:
        start_date = (datetime.now() - timedelta(days=request.historical_days)).strftime('%Y-%m-%d')
        df = db_manager.get_historical_consumption(request.nomenclature_id, start_date)

        if df.empty:
            raise HTTPException(status_code=404, detail="Нет данных для обучения")

        model =

```

```

SkiInventoryForecastModel(model_name=f"ski_forecast_{request.nomenclature_id}")
    model.fit(df)

    import os
    model_path = os.path.join(config.models_dir, f"model_{request.nomenclature_id}.pkl")
    model.save(model_path)
    models_cache[request.nomenclature_id] = model

    return ModelInfoResponse(
        model_name=model.model_name,
        is_trained=True,
        training_metrics=model.training_metrics,
        last_trained=datetime.now().isoformat()
    )

except HTTPException:
    raise
except Exception as e:
    logger.error(f"Ошибка при обучении модели: {e}")
    raise HTTPException(status_code=500, detail=str(e))

@app.get("/model/{nomenclature_id}", response_model=ModelInfoResponse)
async def get_model_info(nomenclature_id: int):
    """Получение информации о модели для номенклатуры."""
    import os
    model_path = os.path.join(config.models_dir, f"model_{nomenclature_id}.pkl")

    if os.path.exists(model_path):
        try:
            model = SkiInventoryForecastModel.load(model_path)
            # Получение времени последнего изменения файла
            import os
            mtime = os.path.getmtime(model_path)
            last_trained = datetime.fromtimestamp(mtime).isoformat()

            return ModelInfoResponse(
                model_name=model.model_name,
                is_trained=True,
                training_metrics=model.training_metrics,
                last_trained=last_trained
            )
        except Exception as e:
            logger.error(f"Ошибка загрузки модели: {e}")

    return ModelInfoResponse(
        model_name=f"ski_forecast_{nomenclature_id}",
        is_trained=False,
        training_metrics={},
        last_trained=None
    )

@app.get("/nomenclature/{nomenclature_id}/history")
async def get_historical_data(
    nomenclature_id: int,
    days: int = Query(365, ge=30, le=1095, description="Количество дней истории")
):
    """Получение исторических данных по расходу инвентаря."""
    start_date = (datetime.now() - timedelta(days=days)).strftime('%Y-%m-
```



```

%d')
    df = db_manager.get_historical_consumption(nomenclature_id, start_date)

    return {
        "nomenclature_id": nomenclature_id,
        "days": days,
        "data": df.to_dict('records') if not df.empty else []
    }

@app.get("/events/upcoming")
async def get_upcoming_events(days: int = Query(90, ge=7, le=365)):
    """Получение предстоящих соревнований и сборов."""
    df = db_manager.get_upcoming_events(days)
    return {
        "days_ahead": days,
        "events": df.to_dict('records') if not df.empty else []
    }

if __name__ == "__main__":
    uvicorn.run(
        "main:app",
        host="0.0.0.0",
        port=8000,
        reload=True,
        log_level=config.log_level.lower()
    )

```

Листинг 8 – Dockerfile

```

FROM python:3.12-slim

WORKDIR /app

# Установка системных зависимостей
RUN apt-get update && apt-get install -y \
    gcc \
    g++ \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

# Установка Python-зависимостей
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Копирование исходного кода
COPY src/ ./src/

# Создание директории для моделей
RUN mkdir -p /app/models

# Переменные окружения
ENV PYTHONPATH=/app
ENV MODELS_DIR=/app/models

EXPOSE 8000

CMD ["uvicorn", "src.api.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

Листинг 9 – Файл *docker-compose.yml* (для интеграции с основной АСУ)

```
version: '3.8'

services:
  forecast-api:
    build: .
    container_name: asu-forecast-api
    ports:
      - "8000:8000"
    environment:
      - DB_HOST=postgres
      - DB_PORT=5432
      - DB_NAME=asu_inventory
      - DB_USERNAME=${ASU_DB_USERNAME}
      - DB_PASSWORD=${ASU_DB_PASSWORD}
      - MODELS_DIR=/app/models
      - LOG_LEVEL=INFO
    volumes:
      - ./models:/app/models
    networks:
      - asu-network
    restart: unless-stopped
    depends_on:
      - postgres

  postgres:
    image: postgres:15
    container_name: asu-postgres
    environment:
      - POSTGRES_DB=asu_inventory
      - POSTGRES_USER=${ASU_DB_USERNAME}
      - POSTGRES_PASSWORD=${ASU_DB_PASSWORD}
    volumes:
      - postgres-data:/var/lib/postgresql/data
    networks:
      - asu-network
    restart: unless-stopped

networks:
  asu-network:
    driver: bridge

volumes:
  postgres-data:
```

Листинг 10 – Файл *notebooks/exploratory_analysis.ipynb*

```
# %% [markdown]
# # Исследовательский анализ данных расхода лыжного инвентаря
#
# Анализ исторических данных для выявления паттернов сезонности и трендов.
#
# %%
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from prophet import Prophet
from prophet.plot import plot_plotly, plot_components_plotly
import plotly.graph_objs as go

# Настройки визуализации
plt.style.use('seaborn-v0_8-darkgrid')
```

```

sns.set_palette("husl")
plt.rcParams['figure.figsize'] = (14, 6)

# %%
# Загрузка данных из БД
import sys
sys.path.append('.')
from src.data.database import DatabaseManager

db = DatabaseManager()
df = db.get_historical_consumption(
    nomenclature_id=123, # ЛЬЖИ ГОНОЧНЫЕ
    start_date='2023-01-01'
)

print(f"Загружено {len(df)} записей")
print(f"Период: {df['ds'].min()} - {df['ds'].max()}")
df.head()

# %%
# Визуализация временного ряда
fig = go.Figure()
fig.add_trace(go.Scatter(
    x=df['ds'],
    y=df['y'],
    mode='lines+markers',
    name='Выдачи инвентаря'
))
fig.update_layout(
    title='Динамика выдач лыжного инвентаря',
    xaxis_title='Дата',
    yaxis_title='Количество',
    height=500
)
fig.show()

# %%
# Анализ сезонности
df['month'] = pd.to_datetime(df['ds']).dt.month
df['year'] = pd.to_datetime(df['ds']).dt.year

monthly_stats = df.groupby('month')['y'].agg(['mean', 'std', 'count']).reset_index()
monthly_stats

# %%
# Визуализация сезонного профиля
fig, ax = plt.subplots(figsize=(12, 6))
months = ['Янв', 'Фев', 'Мар', 'Апр', 'Май', 'Июн',
          'Июл', 'Авг', 'Сен', 'Окт', 'Ноя', 'Дек']
ax.bar(months, monthly_stats['mean'].values)
ax.set_title('Среднемесячное количество выдач инвентаря')
ax.set_xlabel('Месяц')
ax.set_ylabel('Среднее количество')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# %%
# Обучение модели Prophet
model = Prophet(
    yearly_seasonality=True,
    weekly_seasonality=True,
    seasonality_mode='multiplicative'
)

```

```

)
model.add_country_holidays(country_name='RU')
model.fit(df)

# %%
# Прогноз на 90 дней
future = model.make_future_dataframe(periods=90)
forecast = model.predict(future)

# Визуализация прогноза
plot_plotly(model, forecast)

# %%
# Компоненты модели (тренд, сезонность)
plot_components_plotly(model, forecast)

# %%
# Оценка качества модели
from sklearn.metrics import mean_absolute_error, mean_absolute_percentage_error

# Разделение на train/test
train_size = int(len(df) * 0.8)
train_df = df[:train_size]
test_df = df[train_size:]

model_test = Prophet(
    yearly_seasonality=True,
    weekly_seasonality=True,
    seasonality_mode='multiplicative'
)
model_test.fit(train_df)

future_test = model_test.make_future_dataframe(periods=len(test_df))
forecast_test = model_test.predict(future_test)

# Метрики
y_true = test_df['y'].values
y_pred = forecast_test['yhat'].values[-len(test_df):]

mae = mean_absolute_error(y_true, y_pred)
mape = mean_absolute_percentage_error(y_true, y_pred)

print(f"MAE: {mae:.2f}")
print(f"MAPE: {mape:.2%}")

```